

1. Write a program in C to find time complexity of Bubble sort by using RAM Model.

Source code:

```
#include <stdio.h>

int main() {
    int a[100], n, i, j, temp;

    int count = 0; // To count operations (RAM model)

    print("Amir Bhattarai" \n);
    printf("Enter number of elements: ");
    scanf("%d", &n);

    count = count+6;

    printf("Enter elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        count++; // input operation
    }

    // Bubble Sort
    for(i = 0; i < n-1; i++) {
        count++; // loop check
        for(j = 0; j < n-i-1; j++) {
            count++; // comparison
            if(a[j] > a[j+1]) {
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
                count += 3; // swap operations
            }
        }
    }
}
```

```

        count = count+2;

        printf("Sorted array:\n");

        for(i = 0; i < n; i++) {

            count++;

            printf("%d ", a[i]);

        }

        count = count+3;

        printf("\nTotal RAM operations = %d\n", count);

        printf("Time Complexity of Bubble Sort = O(n^2) \n");

        return 0;

    }

```

Output 1:

Output
<pre> Amir Bhattarai Enter number of elements: 5 Enter elements: 234 45 23 67 44 Sorted array: 23 44 45 67 234 Total RAM operations = 56 Time Complexity of Bubble Sort = O(n^2) </pre>

Output 2:

Output
<pre> Amir Bhattarai Enter number of elements: 5 Enter elements: 23 456 11 67 34 Sorted array: 11 23 34 67 456 Total RAM operations = 50 Time Complexity of Bubble Sort = O(n^2) </pre>

Output 3:

Output
<pre> Amir Bhattarai Enter number of elements: 5 Enter elements: 56 43 78 655 23 Sorted array: 23 43 56 78 655 Total RAM operations = 50 Time Complexity of Bubble Sort = O(n^2) </pre>

2. Write a program in c to find time complexity of Insertion sort by using RAM Model.

Source code:

```
#include <stdio.h>

int main() {
    int a[100], n, i, j, key;
    int count = 0;
    printf("Amir Bhattarai \n");
    printf("Enter number of elements: ");
    scanf("%d", &n);
    count += 3;
    printf("Enter elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        count++;
    }
    // Insertion Sort
    for(i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;
        count += 2;
        while(j >= 0 && a[j] > key) {
            a[j+1] = a[j];
            j--;
            count += 2;
        }
        a[j+1] = key;
        count++;
    }
}
```

```

printf("Sorted array:\n");
for(i = 0; i < n; i++) {
    printf("%d ", a[i]);
    count++;
}

printf("\nTotal RAM operations = %d\n", count);
printf("Time Complexity of Insertion Sort = O(n^2) \n");
return 0;
}

```

Output 1:

Output
<pre> Amir Bhattarai Enter number of elements: 5 Enter elements: 564 23 76 233 76 Sorted array: 23 76 76 233 564 Total RAM operations = 35 Time Complexity of Insertion Sort = O(n^2) </pre>

Output 2:

Output
<pre> Amir Bhattarai Enter number of elements: 5 Enter elements: 78 56 23 556 34 Sorted array: 23 34 56 78 556 Total RAM operations = 37 Time Complexity of Insertion Sort = O(n^2) </pre>

Output 3:

Output
<pre> Amir Bhattarai Enter number of elements: 5 Enter elements: 98 67 454 87 23 Sorted array: 23 67 87 98 454 Total RAM operations = 39 Time Complexity of Insertion Sort = O(n^2) </pre>

3. Write a program in c to find time complexity of Insertion sort by using RAM Model.

Source Code:

```
#include <stdio.h>

int main() {
    int a[100], n, i, j, min, temp;
    int count = 0;
    printf("Amir Bhattarai: \n");
    printf("Enter number of elements: ");
    scanf("%d", &n);
    count += 3;
    printf("Enter elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        count++;
    }
    // Selection Sort
    for(i = 0; i < n-1; i++) {
        min = i;
        count++;
        for(j = i+1; j < n; j++) {
            count++;
            if(a[j] < a[min]) {
                min = j;
                count++;
            }
        }
        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
        count += 3;
    }
    printf("Sorted array:\n");
    for(i = 0; i < n; i++) {
        printf("%d ", a[i]);
        count++;
    }
    printf("\nTotal RAM operations = %d\n", count);
    printf("Time Complexity of Selection Sort = O(n^2) \n");

    return 0;
}
```

Output 1:

Output
Amir Bhattarai: Enter number of elements: 5 Enter elements: 78 456 767 234 98 Sorted array: 78 98 234 456 767 Total RAM operations = 43 Time Complexity of Selection Sort = $O(n^2)$

Output 2:

Output
Amir Bhattarai: Enter number of elements: 5 Enter elements: 4 776 23 87 45 Sorted array: 4 23 45 87 776 Total RAM operations = 42 Time Complexity of Selection Sort = $O(n^2)$

Output 3:

Output
Amir Bhattarai: Enter number of elements: 5 Enter elements: 34 67 23 78 54 Sorted array: 23 34 54 67 78 Total RAM operations = 43 Time Complexity of Selection Sort = $O(n^2)$

4. Write a program in c to find GCD of given two numbers.

Source Code:

```
#include <stdio.h>
int main() {
    printf("Amir Bhattarai: \n");
    int a, b, temp;
    int count = 0;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    count += 3;
    // Euclidean Algorithm
    while(b != 0) {
        count++;
        temp = b;
        b = a % b;
        a = temp;
        count += 3;
    }
    count++;
    printf("GCD = %d\n", a);
    printf("Total RAM operations = %d\n", count);

    return 0;
}
```

Output:

Output

```
Amir Bhattarai:
Enter two numbers: 24 12
GCD = 12
Total RAM operations = 8
```

5. Write a program in c to sort given numbers by using merge sort and find execution time by using clock function.

Source code:

```
#include <stdio.h>
#include <time.h>
void merge(int arr[], int left, int mid, int right)
{
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    // Copy data to temporary arrays
    for(i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for(j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
    i = 0;
    j = 0;
    k = left;
    // Merge the temp arrays
    while(i < n1 && j < n2)
    {
        if(L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    // Copy remaining elements
    while(i < n1)
    {
        arr[k] = L[i];
        i++;
        k++;
    }
}
```



```

        while(j < n2)
        {
            arr[k] = R[j];
            j++;
            k++;
        }
    }
}

void mergeSort(int arr[], int left, int right)
{
    if(left < right)
    {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main()
{
    int arr[100], n, i;
    clock_t start, end;
    double cpu_time_used;
    printf("Amir Bhattarai : \n");
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    start = clock(); // Start time
    mergeSort(arr, 0, n - 1);

    end = clock(); // End time
    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nExecution Time: %f seconds\n", cpu_time_used);
    return 0;
}

```

Output 1:

Output
Amir Bhattarai : Enter number of elements: 5 Enter elements: 567 34 8 23 89 Sorted array: 8 23 34 89 567 Execution Time: 0.000003 seconds

Output 2:

Output
Amir Bhattarai : Enter number of elements: 5 Enter elements: 34 67 32 78 54 Sorted array: 32 34 54 67 78 Execution Time: 0.000002 seconds

Output 3:

Output
Amir Bhattarai : Enter number of elements: 5 Enter elements: 34 23 76 45 45 Sorted array: 23 34 45 45 76 Execution Time: 0.000002 seconds

6. Write a program in c to sort given numbers by using Quick sort and find execution time by using clock function.

```
#include<stdio.h>
#include<time.h>
void swap(int *a,int *b){int t=*a;*a=*b;*b=t;}
int part(int a[],int l,int h){
    int p=a[h],i=l-1,j;
    for(j=l;j<h;j++) if(a[j]<p) swap(&a[++i],&a[j]);
    swap(&a[i+1],&a[h]);
    return i+1;
}
void quick(int a[],int l,int h){
    if(l<h){
        int pi=part(a,l,h);
        quick(a,l,pi-1);
        quick(a,pi+1,h);
    }
}
int main(){
    int n,i , a[n];
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter numbers:\n");
    for(i=0;i<n;i++) scanf("%d",&a[i]);
    printf("Amir Bhattarai\n");
    clock_t s=clock();
    quick(a,0,n-1);
    clock_t e=clock();
    printf("Sorted array:\n");
    for(i=0;i<n;i++) printf("%d ",a[i]);
    printf("\nExecution time: %f seconds",(double)(e-s)/CLOCKS_PER_SEC);
}
```

Output 1:

Output
Enter number of elements: 5 Enter numbers: 12 34 564 23 76 Amir Bhattarai Sorted array: 12 23 34 76 564 Execution time: 0.000002 seconds

Output 2:

Output
Enter number of elements: 5 Enter numbers: 56 45 34 67 23 Amir Bhattarai Sorted array: 23 34 45 56 67 Execution time: 0.000003 seconds

Output 3:

Output
Enter number of elements: 5 Enter numbers: 345 67 23 66 12 Amir Bhattarai Sorted array: 12 23 66 67 345 Execution time: 0.000002 seconds

7. Write a program in c to sort given numbers by using Heap sort and find execution time by using clock function.

```
#include<stdio.h>
#include<time.h>
void swap(int *a,int *b){int t=*a;*a=*b;*b=t;}
void heapify(int a[],int n,int i){
    int l=2*i+1,r=2*i+2,largest=i;
    if(l<n && a[l]>a[largest]) largest=l;
    if(r<n && a[r]>a[largest]) largest=r;
    if(largest!=i){
        swap(&a[i],&a[largest]);
        heapify(a,n,largest);
    }
}
void heapSort(int a[],int n){
    for(int i=n/2-1;i>=0;i--) heapify(a,n,i);
    for(int i=n-1;i>0;i--){
        swap(&a[0],&a[i]);
        heapify(a,i,0);
    }
}
int main(){
    int n,i;
    printf("Amir Bhattarai : \n ");
    printf("Enter number of elements: ");
    scanf("%d",&n);
    int a[n];
    printf("Enter numbers:\n");
    for(i=0;i<n;i++) scanf("%d",&a[i]);
    clock_t s=clock();
    heapSort(a,n);
    clock_t e=clock();
```

```
printf("Sorted array:\n");  
for(i=0;i<n;i++) printf("%d ",a[i]);  
printf("\nExecution time: %f seconds",(double)(e-s)/CLOCKS_PER_SEC);  
}
```

Output 1:

Output
Amir Bhattarai : Enter number of elements: 5 Enter numbers: 45 23 65 12 67 Sorted array: 12 23 45 65 67 Execution time: 0.000002 seconds

Output 2:

Output
Amir Bhattarai : Enter number of elements: 5 Enter numbers: 45 23 767 43 76 Sorted array: 23 43 45 76 767 Execution time: 0.000002 seconds

Output 3:

Output
Amir Bhattarai : Enter number of elements: 5 Enter numbers: 23 67 12 567 34 Sorted array: 12 23 34 67 567 Execution time: 0.000003 seconds

8. Write a program in c to sort given numbers by using Randomized Quick sort and find execution time by using clock function.

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
void swap(int *a,int *b){int t=*a;*a=*b;*b=t;}
int part(int a[],int l,int h){
    int p=a[h],i=l-1;
    for(int j=l;j<h;j++)
        if(a[j]<p) swap(&a[++i],&a[j]);
    swap(&a[i+1],&a[h]);
    return i+1;
}
int randPart(int a[],int l,int h){
    int r=l+rand()%(h-l+1);
    swap(&a[r],&a[h]);
    return part(a,l,h);
}
void quick(int a[],int l,int h){
    if(l<h){
        int pi=randPart(a,l,h);
        quick(a,l,pi-1);
        quick(a,pi+1,h);
    }
}
int main(){
    int n,i;
    printf("Amir Bhattarai : \n ");
    printf("Enter number of elements: ");
    scanf("%d",&n);
    int a[n];
    printf("Enter numbers:\n");
```

```

for(i=0;i<n;i++) scanf("%d",&a[i]);
srand(time(0)); // seed for randomness
clock_t s=clock();
quick(a,0,n-1);
clock_t e=clock();
printf("Sorted array:\n");
for(i=0;i<n;i++) printf("%d ",a[i]);
printf("\nExecution time: %f seconds",(double)(e-s)/CLOCKS_PER_SEC);
}

```

Output 1:

Output
<pre> Amir Bhattarai : Enter number of elements: 5 Enter numbers: 12 4 65 34 23 Sorted array: 4 12 23 34 65 Execution time: 0.000003 seconds </pre>

Output 2:

Output
<pre> Amir Bhattarai : Enter number of elements: 5 Enter numbers: 23 56 23 6 45 Sorted array: 6 23 23 45 56 Execution time: 0.000003 seconds </pre>

Output 3:

Output
<pre> Amir Bhattarai : Enter number of elements: 5 Enter numbers: 76 45 34 87 23 Sorted array: 23 34 45 76 87 Execution time: 0.000004 seconds </pre>

9. Write a program in c to sort optimal multiplication cost of given matrices.

```
#include <stdio.h>
#include <limits.h>
int main() {
    int n, i, j, k, l;
    printf("Amir Bhattarai : \n");
    printf("Enter number of matrices: ");
    scanf("%d", &n);
    int p[n + 1], m[n][n];
    printf("Enter dimensions:\n");
    for (i = 0; i <= n; i++)
        scanf("%d", &p[i]);
    for (i = 0; i < n; i++)
        m[i][i] = 0;
    for (l = 2; l <= n; l++) {
        for (i = 0; i < n - l + 1; i++) {
            j = i + l - 1;
            m[i][j] = INT_MAX;
            for (k = i; k < j; k++) {
                int cost = m[i][k] + m[k + 1][j] + p[i] * p[k + 1] * p[j + 1];
                if (cost < m[i][j])
                    m[i][j] = cost;
            }
        }
    }
    printf("Optimal multiplication cost: %d\n", m[0][n - 1]);
    return 0;
}
```

Output 1:

Output

```
Amir Bhattarai :  
Enter number of matrices: 3  
Enter dimensions:  
10 20 30 40  
Optimal multiplication cost: 18000
```

Output 2:

Output

```
Amir Bhattarai :  
Enter number of matrices: 3  
Enter dimensions:  
7 7 7 7  
Optimal multiplication cost: 686
```

Output 3:

Output

```
Amir Bhattarai :  
Enter number of matrices: 2  
Enter dimensions:  
10 01 10  
Optimal multiplication cost: 100
```

10. Write a program in c to sort sequences of jobs and their cost by using job sequencing with deadline.

```
#include <stdio.h>

struct Job {
    int id, d, p;
};

void swap(struct Job *a, struct Job *b) {
    struct Job t = *a;
    *a = *b;
    *b = t;
}

int main() {
    int n, i, j;
    printf("Amir Bhattarai : \n");
    printf("Enter number of jobs: ");
    scanf("%d", &n);
    struct Job a[n];
    printf("Enter job id, deadline and profit:\n");
    for (i = 0; i < n; i++)
        scanf("%d%d%d", &a[i].id, &a[i].d, &a[i].p);
    // sort jobs by profit (descending)
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (a[j].p < a[j + 1].p)
                swap(&a[j], &a[j + 1]);
    int slot[n], res[n];
    for (i = 0; i < n; i++) slot[i] = -1;
    int profit = 0;
    for (i = 0; i < n; i++) {
        for (j = a[i].d - 1; j >= 0; j--) {
            if (slot[j] == -1) {
                slot[j] = 1;
```

```

        res[j] = a[i].id;
        profit += a[i].p;
        break;
    }
}
}

printf("Job sequence:\n");
for (i = 0; i < n; i++)
    if (slot[i] == 1)
        printf("%d ", res[i]);
printf("\nTotal profit: %d", profit);
return 0;
}

```

Output 1:

```

Amir Bhattarai :
Enter number of jobs: 4
Enter job id, deadline and profit:
1 2 100
2 1 19
3 2 27
4 1 25
Job sequence:
3 1
Total profit: 127

```

Output 2:

```

Amir Bhattarai :
Enter number of jobs: 4
Enter job id, deadline and profit:
1 4 70
2 1 80
3 1 30
4 2 100
Job sequence:
2 4 1
Total profit: 250

```

Output 3:

```

Amir Bhattarai :
Enter number of jobs: 6
Enter job id, deadline and profit:
1 2 50
2 1 20
3 2 40
4 1 60
5 3 30
6 2 25
Job sequence:
4 1 5
Total profit: 140

```

11. Write a program in c to find maximum profit by using 0/1 knapsack problem.

```
#include <stdio.h>

int max(int a, int b) { return (a > b) ? a : b; }

int main() {
    int n, W, i, w;
    printf("Amir Bhattarai :\n");
    printf("Enter number of items: ");
    scanf("%d", &n);
    int val[n], wt[n];
    printf("Enter weights of items:\n");
    for (i = 0; i < n; i++) scanf("%d", &wt[i]);
    printf("Enter profits of items:\n");
    for (i = 0; i < n; i++) scanf("%d", &val[i]);
    printf("Enter knapsack capacity: ");
    scanf("%d", &W);
    int K[n + 1][W + 1];
    // Build table K[][] in bottom-up manner
    for (i = 0; i <= n; i++) {
        for (w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                K[i][w] = 0;
            else if (wt[i - 1] <= w)
                K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]], K[i - 1][w]);
            else
                K[i][w] = K[i - 1][w];
        }
    }
    printf("Maximum profit: %d\n", K[n][W]);
    return 0;
}
```

Output 1:

```
Amir Bhattarai :  
Enter number of items: 3  
Enter weights of items:  
10 20 30  
Enter profits of items:  
60 100 120  
Enter knapsack capacity: 50  
Maximum profit: 220
```

Output 2:

```
Amir Bhattarai :  
Enter number of items: 4  
Enter weights of items:  
2 3 4 5  
Enter profits of items:  
3 4 5 6  
Enter knapsack capacity: 5  
Maximum profit: 7
```

Output 3:

```
Amir Bhattarai :  
Enter number of items: 5  
Enter weights of items:  
1 2 3 4 5  
Enter profits of items:  
10 20 30 40 50  
Enter knapsack capacity: 10  
Maximum profit: 100
```

12. Write a program in c to find path compression of given text file by using Huffman algorithm.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 256
struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
};

struct Node* createNode(char ch, int freq) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->ch = ch;
    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}

void swap(struct Node **a, struct Node **b) {
    struct Node* t = *a;
    *a = *b;
    *b = t;
}

void sort(struct Node* arr[], int n) {
    for(int i=0;i<n-1;i++)
        for(int j=0;j<n-i-1;j++)
            if(arr[j]->freq>arr[j+1]->freq)
                swap(&arr[j], &arr[j+1]);
}

struct Node* buildHuffman(struct Node* nodes[], int n) {
    while(n>1) {
        sort(nodes,n);
```

```

    struct Node *l=nodes[0],*r=nodes[1];
    struct Node *top=createNode('\0', l->freq+r->freq);
    top->left=l; top->right=r;
    for(int i=2;i<n;i++) nodes[i-2]=nodes[i];
    nodes[n-2]=top;
    n--;
}
return nodes[0];
}

void printCodes(struct Node* root, char str[], int top) {
    if(root->left) { str[top]='0'; printCodes(root->left,str,top+1); }
    if(root->right){ str[top]='1'; printCodes(root->right,str,top+1); }
    if(!root->left && !root->right) {
        str[top]='\0';
        printf("%c: %s\n", root->ch, str);
    }
}

int main() {
    printf("Amir Bhattarai:\n");
    FILE *f=fopen("input.txt","r");
    if(!f){ printf("File not found\n"); return 1; }
    int freq[MAX]={0};
    char ch;
    while((ch=fgetc(f))!=EOF) freq[(unsigned char)ch]++;
    fclose(f);
    int n=0;
    struct Node* nodes[MAX];
    for(int i=0;i<MAX;i++)
        if(freq[i]>0)
            nodes[n++]=createNode((char)i,freq[i]);

    struct Node* root=buildHuffman(nodes,n);

```



```
    char arr[MAX];  
    printf("Huffman Codes:\n");  
    printCodes(root, arr, 0);  
    return 0;  
}
```

Output :

```
amir@amirnitrov15:~/Documents/daa$ g++ lab12.c  
amir@amirnitrov15:~/Documents/daa$ ./a.out  
Amir Bhattarai:  
Huffman Codes:  
h: 000  
r: 001  
l: 01  
w: 100  
o: 101  
 : 1100  
!: 1101  
d: 1110  
e: 1111
```

13. Write a program in c to find shortest path by using Floyd warshall algorithm (all pair).

```
#include <stdio.h>

#define INF 99999

int main() {
    int n, i, j, k;

    printf("Amir Bhattarai:\n");
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int graph[n][n];

    printf("Enter adjacency matrix (use %d for no edge):\n", INF);
    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    int dist[n][n];

    // initialize distance matrix
    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            dist[i][j] = graph[i][j];

    // Floyd-Warshall main loop
    for(k = 0; k < n; k++)
        for(i = 0; i < n; i++)
            for(j = 0; j < n; j++)
                if(dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];

    printf("All-pairs shortest path matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            if(dist[i][j] == INF)
```

```

        printf("INF ");
    else
        printf("%d ", dist[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Output 1:

```

Amir Bhattarai:
Enter number of vertices: 4
Enter adjacency matrix (use 99999 for no edge):
0 5 9 10
9 0 3 9
9 9 0 1
9 9 9 0
All-pairs shortest path matrix:
0 5 8 9
9 0 3 4
9 9 0 1
9 9 9 0

```

Output 2:

```

Amir Bhattarai:
Enter number of vertices: 3
Enter adjacency matrix (use 99999 for no edge):
0 1 4
9 0 2
9 9 0
All-pairs shortest path matrix:
0 1 3
9 0 2
9 9 0

```

Output 3:

```

Amir Bhattarai:
Enter number of vertices: 3
Enter adjacency matrix (use 99999 for no edge):
0 1 5
8 1 2
8 8 3
All-pairs shortest path matrix:
0 1 3
8 1 2
8 8 3

```

14. Write a program in c to find shortest path by using Dijkstra's algorithm (single source).

```
#include <stdio.h>

#define INF 99999

int minDistance(int dist[], int visited[], int n) {
    int min = INF, min_index;
    for(int v = 0; v < n; v++)
        if(!visited[v] && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    return min_index;
}

int main() {
    int n, src;
    printf("Amir Bhattarai :\n");
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int graph[n][n];
    printf("Enter adjacency matrix:\n");
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            scanf("%d",&graph[i][j]);
    printf("Enter source vertex (0 to %d): ", n-1);
    scanf("%d",&src);
    int dist[n], visited[n];
    for(int i=0;i<n;i++){
        dist[i]=INF;
        visited[i]=0;
    }
    dist[src]=0;
```

```

for(int count=0; count<n-1; count++) {
    int u = minDistance(dist, visited, n);
    visited[u]=1;
    for(int v=0; v<n; v++)
        if(!visited[v] && graph[u][v]!=INF && dist[u]+graph[u][v]<dist[v])
            dist[v]=dist[u]+graph[u][v];
}
printf("Vertex \t Distance from Source %d\n", src);
for(int i=0;i<n;i++)
    printf("%d \t %d\n", i, dist[i]);
return 0;
}

```

Output 1:

```

Amir Bhattarai :
Enter number of vertices: 3
Enter adjacency matrix:
0 1 4
9 0 2
9 9 0
Enter source vertex (0 to 2): 0
Vertex    Distance from Source 0
0         0
1         1
2         3

```

Output 2:

```

Amir Bhattarai :
Enter number of vertices: 3
Enter adjacency matrix:
0 1 5
2 0 2
4 4 0
Enter source vertex (0 to 2): 1
Vertex    Distance from Source 1
0         2
1         0
2         2

```

Output 3:

```

Amir Bhattarai :
Enter number of vertices: 4
Enter adjacency matrix:
0 4 9 10
9 0 3 0
9 9 0 1
9 9 9 0
Enter source vertex (0 to 3): 0
Vertex    Distance from Source 0
0         0
1         4
2         7
3         4

```

15. Write a program in c to find spanning tree of given graph by using Kruskal's Algorithm.

```
#include <stdio.h>
#include <stdlib.h>
struct Edge {
    int u, v, w;
};
int cmp(const void *a, const void *b) {
    return ((struct Edge*)a)->w - ((struct Edge*)b)->w;
}
// Find parent with path compression
int find(int parent[], int i) {
    if(parent[i] != i)
        parent[i] = find(parent, parent[i]);
    return parent[i];
}
// Union two sets
void Union(int parent[], int x, int y) {
    int xroot = find(parent, x);
    int yroot = find(parent, y);
    parent[xroot] = yroot;
}
int main() {
    int V, E;
    printf("Amir Bhattarai:\n");
    printf("Enter number of vertices and edges: ");
    scanf("%d %d", &V, &E);

    struct Edge edges[E];
    printf("Enter edges (u v weight) 0-based indices:\n");
    for(int i=0;i<E;i++)
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
```

```

// Sort edges by weight
qsort(edges, E, sizeof(struct Edge), cmp);
int parent[V];
for(int i=0;i<V;i++) parent[i]=i;
printf("Edges in MST:\n");
int mst_weight = 0;
for(int i=0;i<E;i++) {
    int u = edges[i].u;
    int v = edges[i].v;
    int set_u = find(parent, u);
    int set_v = find(parent, v);
    if(set_u != set_v) {
        printf("%d - %d : %d\n", u, v, edges[i].w);
        mst_weight += edges[i].w;
        Union(parent, set_u, set_v);
    }
}
printf("Total weight of MST: %d\n", mst_weight);
return 0;
}

```

Output 1:

```

Amir Bhattarai:
Enter number of vertices and edges: 4 5
Enter edges (u v weight) 0-based indices:
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
Edges in MST:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Total weight of MST: 19

```

Output 2:

```
Amir Bhattarai:  
Enter number of vertices and edges: 3 3  
Enter edges (u v weight) 0-based indices:  
0 1 1  
1 2 2  
0 2 3  
Edges in MST:  
0 - 1 : 1  
1 - 2 : 2  
Total weight of MST: 3
```

Output 3:

```
Amir Bhattarai:  
Enter number of vertices and edges: 3 3  
Enter edges (u v weight) 0-based indices:  
0 2 2  
1 1 2  
0 1 3  
Edges in MST:  
0 - 2 : 2  
0 - 1 : 3  
Total weight of MST: 5
```